

User Manual

for S32K1_S32M24X QDEC Driver

Document Number: UM2QDECASRR21-11 Rev0000R3.0.0 Rev. 1.0

1 Revision History	2
2 Introduction	3
2.1 Supported Derivatives	3
2.2 Overview	4
2.3 About This Manual	4
2.4 Acronyms and Definitions	5
2.5 Reference List	5
3 Driver	6
3.1 Requirements	6
3.2 Driver Design Summary	6
3.3 Hardware Resources	6
3.4 Deviations from Requirements	6
3.5 Driver Limitations	7
3.6 Driver usage and configuration tips	7
3.7 Runtime errors	7
3.8 Symbolic Names Disclaimer	7
3.9 Usage in non AUTOSAR context	7
4 Configuration Parameters	9
4.1 Qdec	10
4.1.1 Module Qdec	11
4.1.2 Container QdecConfigurationOfOptionalApis	11
4.1.3 Parameter QdecDeInitApi	11
4.1.4 Container QdecInstanceConfig	12
4.1.5 Parameter QdecHardwareModuleId	12
4.1.6 Parameter QdecFtmModule	12
4.1.7 Parameter QdecFtmClockSource	13
4.1.8 Parameter QdecFtmPrescaler	13
4.1.9 Parameter QdecFtmPrescale_Alternate	14
4.1.10 Parameter QdecFtmFilterPrescaler	14
4.1.11 Parameter QdecEncodingMode	15
4.1.12 Parameter QdecOverflowNotification	15
4.1.13 Parameter QdecMinCounterValue	16
4.1.14 Parameter QdecMaxCounterValue	16
4.1.15 Parameter QdecmAPhaseFilter	16
4.1.16 Parameter QdecmAPhaseFilterVal	17
4.1.17 Parameter QdecmAPhasePolarity	17
4.1.18 Parameter QdecmBPhaseFilter	18
4.1.19 Parameter QdecmBPhaseFilterVal	18
4.1.20 Parameter QdecmBPhasePolarity	19
4.1.21 Parameter QdecEnToflsr	19
4.1.22 Reference QdecInstanceEcucPartitionRef	19
4.1.23 Container QdecGeneral	20
4.1.24 Parameter QdecMultiPartitionEnabled	20
4.1.25 Parameter QdecDevErrorDetect	21
4.1.26 Parameter QdecEnableUserModeSupport	21
4.1.27 Parameter QdecEnableDualClockMode	21
4.1.28 Parameter QdecOverFlowNotificationSupported	22
4.1.29 Reference QdecEcucPartitionRef	22
4.1.30 Container CommonPublishedInformation	23
4.1.31 Parameter ArReleaseMajorVersion	23
4.1.32 Parameter ArReleaseMinorVersion	23

4.1.33 Parameter ArReleaseRevisionVersion	24
4.1.34 Parameter ModuleId	24
4.1.35 Parameter SwMajorVersion	25
4.1.36 Parameter SwMinorVersion	25
4.1.37 Parameter SwPatchVersion	26
4.1.38 Parameter VendorApiInfix	26
4.1.39 Parameter VendorId	26
4.2 FTM_QDEC_IP	27
4.2.1 Module Ftm_Qdec_Ip	27
5 Module Index	28
5.1 Software Specification	28
6 Module Documentation	29
6.1 Qdec Driver	29
6.1.1 Detailed Description	29
6.1.2 Data Structure Documentation	30
6.1.2.1 struct Qdec_StateType	30
6.1.2.2 struct Qdec_PhaseParamsType	30
6.1.2.3 struct Qdec_InstanceConfigType	30
6.1.2.4 struct Qdec_ConfigType	31
6.1.3 Macro Definition Documentation	31
6.1.3.1 QDEC_VALIDATE_GLOBAL_CALL	31
6.1.4 Enum Reference	31
6.1.4.1 Qdec_EncodingModeType	31
6.1.4.2 Qdec_PhasePolarityType	32
6.1.4.3 Qdec_CounterStateType	32
6.1.4.4 Qdec_ModeType	32
6.1.5 Function Reference	32
6.1.5.1 Qdec_Init()	33
6.1.5.2 Qdec_DeInit()	33
6.1.5.3 Qdec_GetState()	33
6.1.5.4 Qdec_CounterState()	34
6.1.5.5 Qdec_ResetState()	34
6.1.5.6 Qdec_SetMode()	34
6.2 Ftm_Qdec_Ip Driver	36
6.2.1 Detailed Description	36
6.2.2 Data Structure Documentation	37
6.2.2.1 struct Ftm_Qdec_Ip_CallbackCfgType	37
6.2.2.2 struct Ftm_Qdec_Ip_PhaseParamsType	37
6.2.2.3 struct Ftm_Qdec_Ip_ConfigType	37
6.2.2.4 struct Ftm_Qdec_Ip_StateType	38
6.2.3 Types Reference	38
6.2.3.1 Ftm_Qdec_HwAddrType	38
6.2.3.2 Ftm_Qdec_Ip_CallbackType	38
6.2.4 Enum Reference	38
6.2.4.1 Ftm_Qdec_Ip_ClockSourceType	39
6.2.4.2 Ftm_Qdec_Ip_ClockPrescType	39
6.2.4.3 Ftm_Qdec_Ip_ClockFilterPrescType	40
6.2.4.4 Ftm_Qdec_Ip_EncodingModeType	40
6.2.4.5 Ftm_Qdec_Ip_PhasePolarityType	41
6.2.4.6 Ftm_ModeType	41
6.2.5 Function Reference	41
6.2.5.1 Ftm_Qdec_Ip_Init()	41

6.2.5.2 Ftm_Qdec_Ip_Deinit()	42
6.2.5.3 Ftm_Qdec_Ip_GetState()	42
6.2.5.4 Ftm_Qdec_Ip_CounterState()	42
6.2.5.5 Ftm_Qdec_Ip_ResetState()	43
6.2.5.6 Ftm_Qdec_Ip_SetMode()	43
6.2.6 Variable Documentation	43
6.2.6.1 scpFtmBase	43
6.2.6.2 gQdecFtmInitState	43
6.2.6.3 Ftm_Qdec_Ip_pOverflowIrqCallback	44

Chapter 1

Revision History

Revision	Date	Author	Description
1.0	07.03.2025	NXP RTD Team	S32K1_S32M24X Real-Time Drivers AUTOSAR R21-11 Version 3.0.0

Chapter 2

Introduction

- [Supported Derivatives](#)
- [Overview](#)
- [About This Manual](#)
- [Acronyms and Definitions](#)
- [Reference List](#)

This User Manual describes NXP Semiconductor QDEC driver for S32K1_S32M24X microcontrollers. The QDEC driver configuration parameters and deviations from the specification are described in QDEC Driver chapter of this document. QDEC driver requirements and APIs are vendor-specific.

2.1 Supported Derivatives

The software described in this document is intended to be used with the following microcontroller devices of NXP Semiconductors:

- s32k116_qfn32
- s32k116_lqfp48
- s32k118_lqfp48
- s32k118_lqfp64
- s32k142_lqfp48
- s32k142_lqfp64
- s32k142_lqfp100
- s32k142w_lqfp48
- s32k142w_lqfp64
- s32k144_lqfp48
- s32k144_lqfp64 / MWCT1014S_lqfp64
- s32k144_lqfp100 / MWCT1014S_lqfp100
- s32k144_mapbga100
- s32k144w_lqfp48
- s32k144w_lqfp64

- s32k146_lqfp64
- s32k146_lqfp100 / MWCT1015S_lqfp100
- s32k146_mapbga100 / MWCT1015S_mapbga100
- s32k146_lqfp144
- s32k148_lqfp100
- s32k148_mapbga100 / MWCT1016S_mapbga100
- s32k148_lqfp144
- s32k148_lqfp176
- s32m241_lqfp64
- s32m242_lqfp64
- s32m243_lqfp64
- s32m244_lqfp64

All of the above microcontroller devices are collectively named as S32K1_S32M24X. Note: MWCT part numbers contain NXP confidential IP for Qi Wireless Power

2.2 Overview

AUTOSAR (AUTomotive Open System ARchitecture) is an industry partnership working to establish standards for software interfaces and software modules for automobile electronic control systems.

AUTOSAR:

- paves the way for innovative electronic systems that further improve performance, safety and environmental friendliness.
- is a strong global partnership that creates one common standard: "Cooperate on standards, compete on implementation".
- is a key enabling technology to manage the growing electrics/electronics complexity. It aims to be prepared for the upcoming technologies and to improve cost-efficiency without making any compromise with respect to quality.
- facilitates the exchange and update of software and hardware over the service life of the vehicle.

2.3 About This Manual

This Technical Reference employs the following typographical conventions:

- **Boldface** style: Used for important terms, notes and warnings.
- *Italic* style: Used for code snippets in the text. Note that C language modifiers such "const" or "volatile" are sometimes omitted to improve readability of the presented code.

Notes and warnings are shown as below:

Note

This is a note.

Warning

This is a warning

2.4 Acronyms and Definitions

Term	Definition
API	Application Programming Interface
ASM	Assembler
BSMI	Basic Software Make file Interface
C/CPP	C and C++ Source Code
CS	Chip Select
CTU	Cross Trigger Unit
DMA	Direct Memory Access
DEM	Diagnostic Event Manager
DET	Development Error Tracer
ECU	Electronic Control Unit
FIFO	First In First Out
LSB	Least Significant Bit
MCU	Micro Controller Unit
MIDE	Multi Integrated Development Environment
MSB	Most Significant Bit
N/A	Not Applicable
QDEC	Quadrature Decoder
RAM	Random Access Memory
SIU	Systems Integration Unit
SWS	Software Specification
VLE	Variable Length Encoding
XML	Extensible Markup Language

2.5 Reference List

#	Title	Version
1	S32K1xx Reference Manual	Rev.14.1, 01/2024
2	S32M24x Reference Manual	Rev. 5, 12/2024
3	S32K1xx Data Sheet	Rev. 14, 08/2021
4	S32M2xx Data Sheet	Rev. 7, 12/2024
5	S32K116 Errata	Mask Set Errata for Mask 0N96V Rev. 17/JAN/2024, 1/2024
6	S32K118 Errata	Mask Set Errata for Mask 0N97V Rev. 15/JAN/2024, 1/2024
7	S32K142 Errata	Mask Set Errata for Mask 0N33V Rev. 15/JAN/2024, 1/2024
8	S32K144 Errata	Mask Set Errata for Mask 0N57U Rev. 15/JAN/2024, 1/2024
9	S32K144W Errata	Mask Set Errata for Mask 0P64A Rev. 03/JUL/2024, 7/2024
10	S32K146 Errata	Mask Set Errata for Mask 0N73V Rev. 15/JAN/2024, 1/2024
11	S32K148 Errata	Mask Set Errata for Mask 0N20V Rev. 15/JAN/2024, 1/2024
12	S32M242 Errata	Mask Set Errata for Mask N33V+P69K Rev.1, 8/2024
13	S32M244 Errata	Mask Set Errata for Mask P64A+P69K Rev.1, 8/2024

Chapter 3

Driver

- [Requirements](#)
- [Driver Design Summary](#)
- [Hardware Resources](#)
- [Deviations from Requirements](#)
- [Driver Limitations](#)
- [Driver usage and configuration tips](#)
- [Runtime errors](#)
- [Symbolic Names Disclaimer](#)
- [Usage in non AUTOSAR context](#)

3.1 Requirements

Requirements for this driver are detailed in the Autosar Driver Software Specification document (See [Table Reference List](#)).

For CDD: QDEC Driver is a Complex Device Driver (CDD), so there are no AUTOSAR requirements regarding this module.

It has vendor-specific requirements and implementation.

3.2 Driver Design Summary

Quadrature decoder with input filters, relative position counting, and interrupt on position count or capture of position count on external event

3.3 Hardware Resources

#	Hardware IP	Description
1	FTM	FlexTimer

3.4 Deviations from Requirements

None.

3.5 Driver Limitations

The QDEC driver software have some following limitations for RTD S32K1 : None

3.6 Driver usage and configuration tips

In this chapter, the extra features from our drivers that are not described in the AutoSAR standard are detailed.

3.7 Runtime errors

No runtime errors used.

3.8 Symbolic Names Disclaimer

All containers having symbolicNameValue set to TRUE in the AUTOSAR schema will generate defines like: `#define <Mip>Conf_<Container_ShortName>_<Container_ID>`

For this reason it is forbidden to duplicate the names of such containers across the RTD configurations or to use names that may trigger other compile issues (e.g. match existing `#ifdefs` arguments).

3.9 Usage in non AUTOSAR context

For the non-ASR environments, both High-Level and IP layer interfaces are exposed and can be used by software units above. These may consist of either higher level NXP SW deliverables, like stacks and middleware, or the application code itself.

The goal of the product is to be versatile, easy to use and integrate with other SW environments, offering both standardized and optimized low-level interfaces.

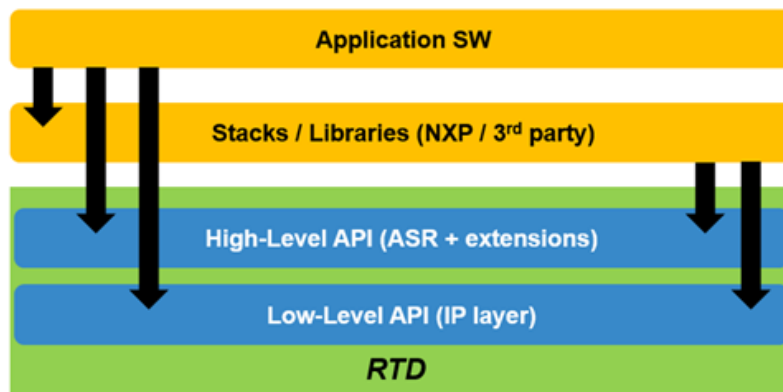


Figure 3.1 RTD Usage in non-ASR Context

In the non-AUTOSAR environment, there are no specific ECU states defined, but an external assumption for the application is that the correct sequence of initialization takes place prior to calling any runtime APIs.

The constraints considered for defining the APIs in the non-AUTOSAR environment are:

- Hardware resources used by the drivers must be properly initialized prior to runtime usage by calling the initialization function with configuration parameters generated by the configuration tools
- All dependencies must be initialized prior to the usage of a driver that depends on them; this translates to a general initialization sequence applicable for all applications, as follows:

1. Initialization of the device clock tree so other periphery can be initialized
2. Configuration of the pinout so peripherals with external connections can communicate with the outside world
3. Configuration of the common resources (e.g. DMA channels, memory regions, access rights, etc) prior to the usage of any runtime functionality that has to benefit from this overall SoC configuration
4. Initialization of specific drivers
5. Runtime functionality
6. Deinitialization (inverse initialization order)

High-level and IP layer functionalities cannot be used simultaneously over the same hardware resource. The granularity of the resource must be defined specifically for each driver, but as a general rule the default granularity considered is the peripheral instance (e.g. Gpt high level APIs cannot be called interchangeably with Pit_Ip low level APIs over the same PIT hardware instance).

The diagram below shows a generic, high-level sequence of calls to RTD interfaces in the non-AUTOSAR scenario:

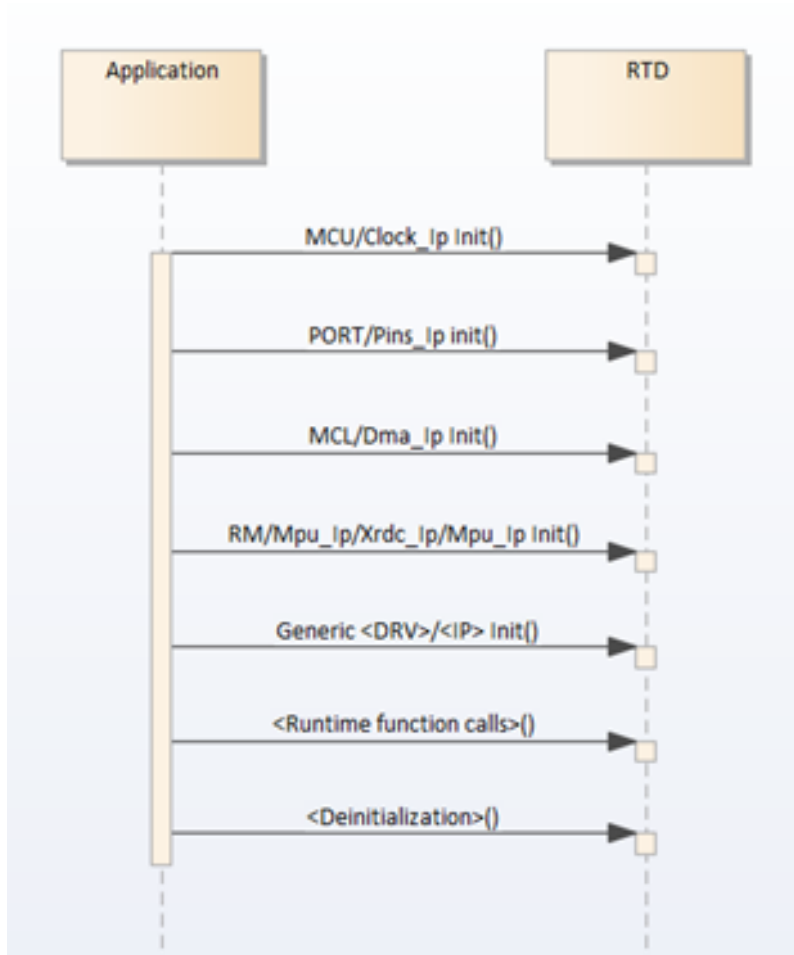


Figure 3.2 Non-AUTOSAR Sequence diagram

Chapter 4

Configuration Parameters

RTD provides configuration schema for the high level driver in EB Tresos proprietary format (**.xdm**), S32CT proprietary format (**.component**), but also in a standardized AUTOSAR format (**.epd**). For the low level driver configuration only the S32CT proprietary format (**.component**) is supported. The configuration data of the project can be saved in EB Tresos proprietary format (**xdm**) and AUTOSAR standardized format (**.epc**) from EB Tresos and in S32CT proprietary format (**.mex**) and AUTOSAR standardized format (**.epc**) from S32 Design Studio. RTD provides templates for parsing the configuration data and generating configuration structures from it in XPath format, to be used with the EB Tresos generator and in JS format, to be used with the S32CT generator. Since the generation process is not standardized by AUTOSAR, only the two mentioned tools can be used with the provided templates. Using the same input configuration data, the same configuration structures will be obtained using any of the generators.

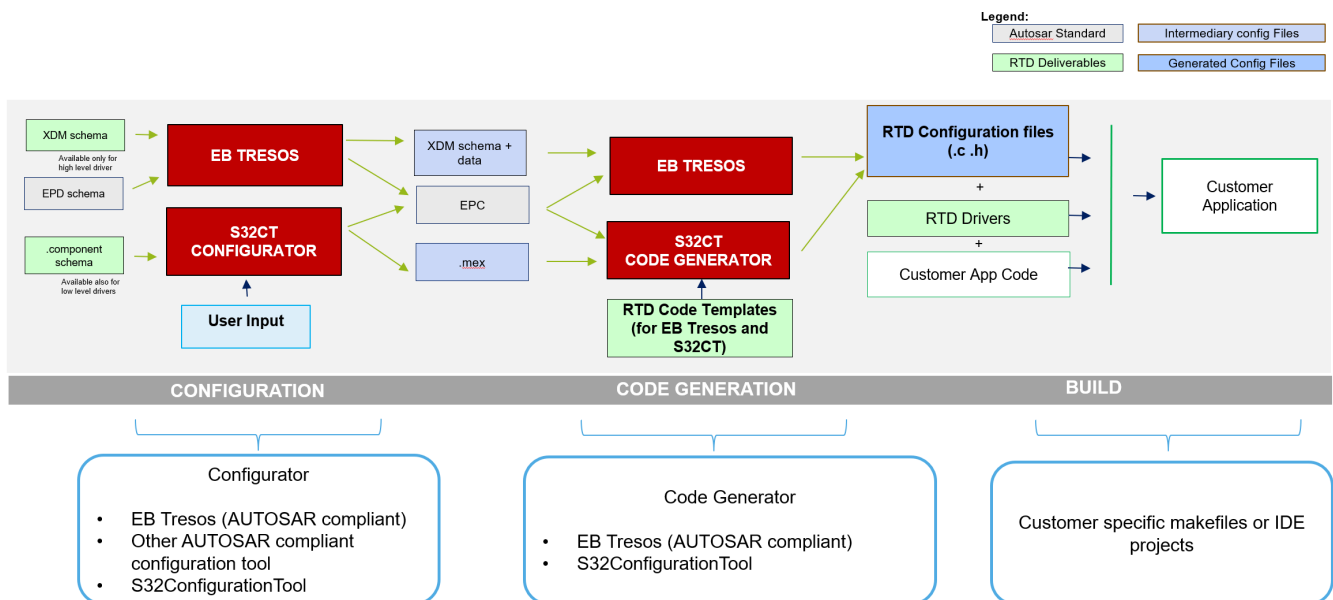


Figure 4.1 Configuration generation flow

- [Qdec](#)
- [FTM_QDEC_IP](#)

4.1 Qdec

- Module [Qdec](#)
 - Container [QdecConfigurationOfOptionalApis](#)
 - * Parameter [QdecDeInitApi](#)
 - Container [QdecInstanceConfig](#)
 - * Parameter [QdecHardwareModuleId](#)
 - * Parameter [QdecFtmModule](#)
 - * Parameter [QdecFtmClockSource](#)
 - * Parameter [QdecFtmPrescaler](#)
 - * Parameter [QdecFtmPrescale__Alternate](#)
 - * Parameter [QdecFtmFilterPrescaler](#)
 - * Parameter [QdecEncodingMode](#)
 - * Parameter [QdecOverflowNotification](#)
 - * Parameter [QdecMinCounterValue](#)
 - * Parameter [QdecMaxCounterValue](#)
 - * Parameter [QdecEnAPhaseFilter](#)
 - * Parameter [QdecmAPhaseFilterVal](#)
 - * Parameter [QdecmAPhasePolarity](#)
 - * Parameter [QdecEnBPhaseFilter](#)
 - * Parameter [QdecmBPhaseFilterVal](#)
 - * Parameter [QdecmBPhasePolarity](#)
 - * Parameter [QdecEnTofIsr](#)
 - * Reference [QdecInstanceEcucPartitionRef](#)
 - Container [QdecGeneral](#)
 - * Parameter [QdecMultiPartitionEnabled](#)
 - * Parameter [QdecDevErrorDetect](#)
 - * Parameter [QdecEnableUserModeSupport](#)
 - * Parameter [QdecEnableDualClockMode](#)
 - * Parameter [QdecOverFlowNotificationSupported](#)
 - * Reference [QdecEcucPartitionRef](#)
 - Container [CommonPublishedInformation](#)
 - * Parameter [ArReleaseMajorVersion](#)
 - * Parameter [ArReleaseMinorVersion](#)
 - * Parameter [ArReleaseRevisionVersion](#)
 - * Parameter [ModuleId](#)
 - * Parameter [SwMajorVersion](#)

- * Parameter [SwMinorVersion](#)
- * Parameter [SwPatchVersion](#)
- * Parameter [VendorApiInfix](#)
- * Parameter [VendorId](#)

4.1.1 Module Qdec

Configuration of QDEC (Quadrature Decoder) module.

Included containers:

- [QdecConfigurationOfOptionalApis](#)
- [QdecInstanceConfig](#)
- [QdecGeneral](#)
- [CommonPublishedInformation](#)

Property	Value
type	ECUC-MODULE-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantSupport	true
supportedConfigVariants	VARIANT-POST-BUILD, VARIANT-PRE-COMPILE

4.1.2 Container QdecConfigurationOfOptionalApis

Configuration of optional APIs.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.3 Parameter QdecDeInitApi

Adds / removes the service `Qdec_DeInit()` from the code.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1

Property	Value
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	true

4.1.4 Container QdecInstanceConfig

This container contains Qdec-specific parameters for selecting optional settings.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	0
upperMultiplicity	Infinite
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE

4.1.5 Parameter QdecHardwareModuleId

ID of the QDEC hardware module.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	0
max	2
min	0

4.1.6 Parameter QdecFtmModule

Selects one Ftm modules available on the platform.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
default Value	FTM_1
literals	['FTM_1', 'FTM_2']

4.1.7 Parameter QdecFtmClockSource

Select origin of clock source used by current instance of FTM.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
default Value	SYSTEM_CLOCK
literals	['SYSTEM_CLOCK', 'EXTERNAL_CLOCK', 'FIXED_FREQUENCY_CLOCK']

4.1.8 Parameter QdecFtmPrescaler

Optional QDEC driver specific clock prescale factor, if supported by hardware. Implementation is defined vendor specific.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE

Configuration Parameters

Property	Value
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	DIV1
literals	['DIV1', 'DIV2', 'DIV4', 'DIV8', 'DIV16', 'DIV32', 'DIV64', 'DIV128']

4.1.9 Parameter QdecFtmPrescale__Alternate

Select input count source (clock) used for this Ftm module. This parameter will be used by the Qdec_SetClockMode function.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	False
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD
defaultValue	DIV1
literals	['DIV1', 'DIV2', 'DIV4', 'DIV8', 'DIV16', 'DIV32', 'DIV64', 'DIV128']

4.1.10 Parameter QdecFtmFilterPrescaler

QDEC driver selects the clock prescaler used in the FTM filters.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	DIV1
literals	['DIV1', 'DIV2', 'DIV3', 'DIV4', 'DIV5', 'DIV6', 'DIV7', 'DIV8', 'DIV9', 'DIV10', 'DIV11', 'DIV12', 'DIV13', 'DIV14', 'DIV15', 'DIV16']

4.1.11 Parameter QdecEncodingMode

Encoding mode used by FTM in the QDEC mode.

-> *MODE_PHASE_ENCODE* < Phase A and phase B encoding mode >

-> *MODE_COUNT_AND_DIR_ENCODE* < Count and direction encoding mode >

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	MODE_COUNT_AND_DIR_ENCODE
literals	['MODE_PHASE_ENCODE', 'MODE_COUNT_AND_DIR_ENCODE']

4.1.12 Parameter QdecOverflowNotification

User callback notification function.

This option is only activated when *QdecGeneral/QdecNotificationSupported* is checked.

<note>

Use *NULL_PTR* without any quotes to determine no notification function is used.

If the string is different from above, it will be used as the notification function name.

</note>

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	NULL_PTR

4.1.13 Parameter QdecMinCounterValue

Minimum counter value.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	65534
min	0

4.1.14 Parameter QdecMaxCounterValue

Maximum counter value.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	65535
max	65535
min	1

4.1.15 Parameter QdecEnAPhaseFilter

Enables the filter for the quadrature decoder phase A,B inputs

False: disable phase filter

True : enable phase filter

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF

Property	Value
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.1.16 Parameter QdecmAPhaseFilterVal

Filter value (if input filter is enabled)

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	15
min	0

4.1.17 Parameter QdecmAPhasePolarity

PHASE_NORMAL Phase input signal is not inverted before identifying the rising and falling edges of this signal

PHASE_INVERT Phase input signal is inverted before identifying the rising and falling edges of this signal

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true

Configuration Parameters

Property	Value
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	PHASE_NORMAL
literals	['PHASE_NORMAL', 'PHASE_INVERT']

4.1.18 Parameter QdecEnBPhaseFilter

Enables the filter for the quadrature decoder phase A,B inputs

False: disable phase filter

True : enable phase filter

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.1.19 Parameter QdecMBPhaseFilterVal

Filter value (if input filter is enabled)

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	15
min	0

4.1.20 Parameter QdecMBPhasePolarity

PHASE_NORMAL Phase input signal is not inverted before identifying the rising and falling edges of this signal

PHASE_INVERT Phase input signal is inverted before identifying the rising and falling edges of this signal

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD
default Value	PHASE_NORMAL
literals	['PHASE_NORMAL', 'PHASE_INVERT']

4.1.21 Parameter QdecEnTofIsr

True: Timer Overflow Interrupt is enabled.

False: Timer Overflow Interrupt is disabled.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	false

4.1.22 Reference QdecInstanceEcucPartitionRef

Maps one or multiple QDEC module to an ECUC partition.

The ECUC partitions referenced are a subset of the ECUC partitions where the QDEC driver is mapped to.

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP

Property	Value
lowerMultiplicity	0
upperMultiplicity	Infinite
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcuDefs/EcuC/EcuPartitionCollection/EcuPartition

4.1.23 Container QdecGeneral

This container contains the module-wide configuration parameters of the QDEC Driver.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.24 Parameter QdecMultiPartitionEnabled

MultiPartition Support Enable.

User can choose ENABLE multi-partition feature by checking this option, this will force to configure at least 1 ECUC partition in QdecInstanceEcuPartitionRef, and each QDEC channel in QdecInstance to configure at least 1 ECUC partition reference in QdecInstanceEcuPartitionRef container to fulfill generating code condition; OR unchecked this option to DISABLE multi-partition feature, performing this action will force user to remove all ECUC partition reference in every Qdec instance contained in QdecInstace and in QdecInstanceEcuPartitionRef.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false

Property	Value
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.1.25 Parameter QdecDevErrorDetect

Enables/Disables development error detection.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.1.26 Parameter QdecEnableUserModeSupport

Enable or disable user mode support for quadrature decoder.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.1.27 Parameter QdecEnableDualClockMode

Enables prescaler settings at mode transition.

true: Enabled.

false: Disabled.

Configuration Parameters

Note: This feature is not required by Autosar.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
default Value	false

4.1.28 Parameter QdecOverflowNotificationSupported

Switch to indicate that the notifications are supported.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
default Value	false

4.1.29 Reference QdecEcucPartitionRef

Maps the QDEC driver to zero or multiple ECUC partitions to make the driver API available in the according partition.

Depending on the addressed timer resource the interfaces operate as follows.

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	0
upperMultiplicity	Infinite
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE

Property	Value
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcuDefs/EcuC/EcuPartitionCollection/EcuPartition

4.1.30 Container CommonPublishedInformation

Common container, aggregated by all modules. It contains published information about vendor and versions.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.31 Parameter ArReleaseMajorVersion

Major version number of AUTOSAR specification on which the appropriate implementation is based on.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	4
max	4
min	4

4.1.32 Parameter ArReleaseMinorVersion

Minor version number of AUTOSAR specification on which the appropriate implementation is based on.

Property	Value
type	ECUC-INTEGER-PARAM-DEF

Configuration Parameters

Property	Value
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	7
max	7
min	7

4.1.33 Parameter ArReleaseRevisionVersion

Revision version number of AUTOSAR specification on which the appropriate implementation is based on.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	0
max	0
min	0

4.1.34 Parameter ModuleId

Module ID of this module from Module List.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

Property	Value
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	255
max	255
min	255

4.1.35 Parameter SwMajorVersion

Major version number of the vendor specific implementation of the module. The numbering is vendor specific.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	3
max	3
min	3

4.1.36 Parameter SwMinorVersion

Minor version number of the vendor specific implementation of the module. The numbering is vendor specific.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	0
max	0
min	0

4.1.37 Parameter SwPatchVersion

Patch level version number of the vendor specific implementation of the module. The numbering is endor specific.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	0
max	0
min	0

4.1.38 Parameter VendorApiInfix

In driver modules which can be instantiated several times on a single ECU, BSW00347 requires that the name of APIs is extended by the VendorId and a vendor specific name.

This parameter is used to specify the vendor specific name. In total, the implementation specific name is generated as follows:

`<ModuleName>_<VendorId>_<VendorApiInfix>`.

E.g. assuming that the VendorId of the implementor is 123 and the implementer chose a VendorApiInfix of "v11r456" a api name Can_Write defined in the SWS will translate to Can_123_v11r456Write.

This parameter is mandatory for all modules with upper multiplicity > 1. It shall not be used for modules with upper multiplicity =1.

Property	Value
type	ECUC-STRING-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	

4.1.39 Parameter VendorId

Vendor ID of the dedicated implementation of this module according to the AUTOSAR vendor list.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	43
max	43
min	43

4.2 FTM_QDEC_IP

The low level driver component can only be used with S32CT Configurator in S32 Design Studio.

- Module [Ftm_Qdec_Ip](#)
 - Container [QdecGeneral](#)
 - * Parameter [QdecDevErrorDetect](#)
 - * Parameter [QdecEnableUserModeSupport](#)
 - * Parameter [QdecEnableDualClockMode](#)
 - * Parameter [QdecOverFlowNotificationSupported](#)
 - Container [QdecInstanceConfig](#)

4.2.1 Module Ftm_Qdec_Ip

Configuration of Ftm_Qdec_Ip (Quadrature Decoder) module.

Included containers:

- [QdecGeneral](#)
- [QdecInstanceConfig](#)

Property	Value
type	MODULE-DEF
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantSupport	true
supportedConfigVariants	VARIANT-POST-BUILD;VARIANT-PRE-COMPILE



Chapter 5

Module Index

5.1 Software Specification

Here is a list of all modules:

Qdec Driver	29
Ftm_Qdec_Ip Driver	36

Chapter 6

Module Documentation

6.1 Qdec Driver

6.1.1 Detailed Description

Data Structures

- struct [Qdec_StateType](#)
Quadrature decode state (counter value and flags) [More...](#)
- struct [Qdec_PhaseParamsType](#)
Quadrature decoder channel parameters. [More...](#)
- struct [Qdec_InstanceConfigType](#)
Quadrature instance configuration. [More...](#)
- struct [Qdec_ConfigType](#)
Qdec high level configuration structure. [More...](#)

Macros

- `#define QDEC\_VALIDATE\_GLOBAL\_CALL`

Enum Reference

- enum [Qdec_EncodingModeType](#)
Encoding mode used in the Quadrature Decoder Mode.
- enum [Qdec_PhasePolarityType](#)
Quadrature phase polarities, normal or inverted polarity.
- enum [Qdec_CounterStateType](#)
Qdec counter state type.
- enum [Qdec_ModeType](#)
Qdec mode type include: `QDEC_MODE_NORMAL` and `QDEC_MODE_SLEEP`.

Function Reference

- void [Qdec_Init](#) (const [Qdec_ConfigType](#) *Config)
This function it is used to initialize a QDEC instance.
- void [Qdec_DeInit](#) (void)

This function it is used to disable QDEC Mode.

- [Qdec_StateType Qdec_GetState](#) (const uint8 module)

Return the current quadrature decoder state (counter value and flags).

- void [Qdec_CounterState](#) (const uint8 module, [Qdec_CounterStateType](#) counterState)

This function will set the state of counter.

- void [Qdec_ResetState](#) (const uint8 module)

Reset state of module to default value.

- void [Qdec_SetMode](#) (const uint8 module, [Qdec_ModeType](#) mode)

Set mode.

6.1.2 Data Structure Documentation

6.1.2.1 struct Qdec_StateType

Quadrature decode state (counter value and flags)

Definition at line 178 of file Qdec_Types.h.

Data Fields

uint32	mCounter	Counter value
boolean	mTimerOverflow	True if timer overflow occurred, False otherwise
boolean	mTimerOverflowDir	Timer Overflow Direction: False if there was a counter decrement and counter changes from its minimum value to its maximum value, True if there was a counter increment and counter changes from its maximum value to its minimum value
boolean	mCountDir	Counter Direction in QDEC mode: False if counting direction is decreasing (counter decrement), True if counting direction is increasing (counter increment)
boolean	mOverRun	True if overrun has occurred, False otherwise
boolean	mMatchEv	True if a match event in the comparators occurred, False otherwise

6.1.2.2 struct Qdec_PhaseParamsType

Quadrature decoder channel parameters.

Definition at line 195 of file Qdec_Types.h.

Data Fields

boolean	mEnPhaseFilter	Enables the filter for the quadrature decoder phase A,B inputs False: disable phase filter, True : enable phase filter
uint8	mPhaseFilterVal	Filter value (if input filter is enabled)
Qdec_PhasePolarityType	mPhasePolarity	Phase polarity

6.1.2.3 struct Qdec_InstanceConfigType

Quadrature instance configuration.

Definition at line 215 of file Qdec_Types.h.

Data Fields

Qdec_EncodingModeType	mEncMode	MODE_PHASE_ENCODE or MODE_COUNT_AND_DIR_ENCODE
uint16	mMinCntVal	Minimum counter value
uint16	mMaxCntVal	Maximum counter value
boolean	mEnToflsr	True: Timer Overflow Interrupt is enabled, False: Timer Overflow Interrupt is disabled
Qdec_PhaseParamsType	mPhaseAConfig	Configuration of input phase A
Qdec_PhaseParamsType	mPhaseBConfig	Configuration of input phase B

6.1.2.4 struct Qdec_ConfigType

Qdec high level configuration structure.

Definition at line 232 of file Qdec_Types.h.

Data Fields

uint8	PartitionIndex	Partition Index
uint8	Instances	Number of QDEC instances used
const Qdec_InstanceConfigType (*	InstanceConfigType)[]	Pointer to user configuration(HLD)
const Qdec_Ipw_ConfigType *	Ipw_ConfigType	Pointer to wrapper layer

6.1.3 Macro Definition Documentation**6.1.3.1 QDEC_VALIDATE_GLOBAL_CALL**

```
#define QDEC_VALIDATE_GLOBAL_CALL
```

Errors and exceptions that will be detected by the QDEC driver.

Definition at line 74 of file Qdec_EnvCfg.h.

6.1.4 Enum Reference**6.1.4.1 Qdec_EncodingModeType**

```
enum Qdec\_EncodingModeType
```

Encoding mode used in the Quadrature Decoder Mode.

Enumerator

MODE_NOT_INITIALIZED	driver is not initialized
MODE_PHASE_ENCODE	Phase A and phase B encoding mode
MODE_COUNT_AND_DIR_ENCODE	Count and direction encoding mode

Definition at line 119 of file Qdec_Types.h.

6.1.4.2 Qdec_PhasePolarityType

enum [Qdec_PhasePolarityType](#)

Quadrature phase polarities, normal or inverted polarity.

Enumerator

PHASE_NORMAL	Phase input signal is not inverted before identifying the rising and falling edges of this signal
PHASE_INVERT	Phase input signal is inverted before identifying the rising and falling edges of this signal

Definition at line 129 of file Qdec_Types.h.

6.1.4.3 Qdec_CounterStateType

enum [Qdec_CounterStateType](#)

Qdec counter state type.

Enumerator

QDEC_RESUME_COUNTER	When the counting starts again to count
QDEC_PAUSE_COUNTER	When counting is paused

Definition at line 154 of file Qdec_Types.h.

6.1.4.4 Qdec_ModeType

enum [Qdec_ModeType](#)

Qdec mode type include: QDEC_MODE_NORMAL and QDEC_MODE_SLEEP.

Enumerator

QDEC_MODE_NORMAL	Normal operation, all used interrupts are enabled according to the notification requests.
QDEC_MODE_SLEEP	Reduced power operation.

Definition at line 165 of file Qdec_Types.h.

6.1.5 Function Reference

6.1.5.1 Qdec_Init()

```
void Qdec_Init (
    const Qdec_ConfigType * Config )
```

This function it is used to initialize a QDEC instance.

This function activates your Quadrature Decoder Mode on the selected module.

Parameters

in	<i>Config</i>	- Qdec configuration structure.
----	---------------	---------------------------------

Returns

void

Precondition

None.

6.1.5.2 Qdec_DeInit()

```
void Qdec_DeInit (
    void )
```

This function it is used to disable QDEC Mode.

This function disable your Quadrature Decoder Mode on the selected module.

Returns

void

Precondition

None.

6.1.5.3 Qdec_GetState()

```
Qdec_StateType Qdec_GetState (
    const uint8 module )
```

Return the current quadrature decoder state (counter value and flags).

This function get the current quadrature decoder state (counter value and flags).

Parameters

in	<i>module</i>	- Name of your QDEC instance.
----	---------------	-------------------------------

Returns

[Qdec_StateType](#)

Precondition

None.

6.1.5.4 Qdec_CounterState()

```
void Qdec_CounterState (
    const uint8 module,
    Qdec_CounterStateType counterState )
```

This function will set the state of counter.

This function will set the state of counter in 2 states:

- PAUSE when counting is paused.
- RESUME when the counting starts again to count.

Parameters

in	<i>module</i>	Name of your QDEC instance.
in	<i>counterState</i>	State of counter.

Returns

void

Precondition

None.

6.1.5.5 Qdec_ResetState()

```
void Qdec_ResetState (
    const uint8 module )
```

Reset state of module to default value.

Parameters

in	<i>module</i>	- Qdec instance ID.
----	---------------	---------------------

Returns

void

Precondition

None.

6.1.5.6 Qdec_SetMode()

```
void Qdec_SetMode (
    const uint8 module,
    Qdec_ModeType mode )
```

Set mode.

Normal operation, all used interrupts are enabled according to the notification requests. Reduced power operation.

Parameters

in	<i>module</i>	- Qdec instance ID. mode - select QDEC_MODE_NORMAL or QDEC_MODE_SLEEP
----	---------------	---

Returns

void

Precondition

None.

6.2 Ftm_Qdec_Ip Driver

6.2.1 Detailed Description

Data Structures

- struct [Ftm_Qdec_Ip_CallbackCfgType](#)
Structure for notification. [More...](#)
- struct [Ftm_Qdec_Ip_PhaseParamsType](#)
Quadrature decoder channel parameters. [More...](#)
- struct [Ftm_Qdec_Ip_ConfigType](#)
Quadrature configure structure. [More...](#)
- struct [Ftm_Qdec_Ip_StateType](#)
FTM Quadrature state (counter value and flags) [More...](#)

Types Reference

- typedef FTM_Type [Ftm_Qdec_HwAddrType](#)
FTM - Register Layout Typedef.
- typedef void(* [Ftm_Qdec_Ip_CallbackType](#)) (uint8 callbackParam1)
Callback type for each instance.

Enum Reference

- enum [Ftm_Qdec_Ip_ClockSourceType](#)
FlexTimer Clock Source Selection.
- enum [Ftm_Qdec_Ip_ClockPrescType](#)
FlexTimer Prescale Factor Selections bits (PS).
- enum [Ftm_Qdec_Ip_ClockFilterPrescType](#)
FlexTimer Filter Prescaler.
- enum [Ftm_Qdec_Ip_EncodingModeType](#)
Encoding mode used by FTM in the QDEC mode.
- enum [Ftm_Qdec_Ip_PhasePolarityType](#)
Quadrature phase polarities, normal or inverted polarity.
- enum [Ftm_ModeType](#)
Qdec mode type include: QDEC_MODE_NORMAL and QDEC_MODE_SLEEP.

Function Reference

- void [Ftm_Qdec_Ip_Init](#) (uint8 module, const [Ftm_Qdec_Ip_ConfigType](#) *config)
Configures the quadrature mode and starts measurement.
- void [Ftm_Qdec_Ip_Deinit](#) (uint8 module)
De-activates the Quadrature Decoder Mode.
- [Ftm_Qdec_Ip_StateType](#) [Ftm_Qdec_Ip_GetState](#) (uint8 module)
Return the current quadrature decoder state (counter value, timer overflow, TOF direction and counting direction flags)

- void [Ftm_Qdec_Ip_CounterState](#) (const uint8 module)
Set state of counter.
- void [Ftm_Qdec_Ip_ResetState](#) (const uint8 module)
Reset state of module to default value.
- void [Ftm_Qdec_Ip_SetMode](#) (const uint8 module, [Ftm_ModeType](#) mode)
Set mode of Qdec module.

Variables

- [Ftm_Qdec_HwAddrType](#) *const [scpFtmBase](#) [FTM_INSTANCE_COUNT]
Array with base addresses for FTM instances available on platform.
- boolean [gQdecFtmInitState](#)
Quadrature ftm initialization state variable.
- [Ftm_Qdec_Ip_CallbackCfgType](#) [Ftm_Qdec_Ip_pOverflowIrqCallback](#) [FTM_INSTANCE_COUNT]
Array with notification handlers for all configurable instances.

6.2.2 Data Structure Documentation

6.2.2.1 struct Ftm_Qdec_Ip_CallbackCfgType

Structure for notification.

The structure used to notification

Definition at line 236 of file [Ftm_Qdec_Ip_Types.h](#).

6.2.2.2 struct Ftm_Qdec_Ip_PhaseParamsType

Quadrature decoder channel parameters.

Definition at line 245 of file [Ftm_Qdec_Ip_Types.h](#).

Data Fields

boolean	mEnPhaseFilter	Enables the filter for the quadrature decoder phase A,B inputs False: disable phase filter, True : enable phase filter
uint8	mPhaseFilterVal	Filter value (if input filter is enabled)
Ftm_Qdec_Ip_PhasePolarityType	mPhasePolarity	Phase polarity

6.2.2.3 struct Ftm_Qdec_Ip_ConfigType

Quadrature configure structure.

Definition at line 257 of file [Ftm_Qdec_Ip_Types.h](#).

Data Fields

Ftm_Qdec_Ip_EncodingModeType	mEncMode	FTM_MODE_PHASE_ENCODE or FTM_MODE_COUNT_AND_DIR_ENCODE
uint16	mMinCntVal	Minimum counter value
uint16	mModCntVal	Maximum counter value

Data Fields

boolean	mEnToflsr	True: Timer Overflow Interrupt is enabled, False: Timer Overflow Interrupt is disabled
Ftm_Qdec_Ip_PhaseParamsType	mPhaseAConfig	Configuration of input phase A
Ftm_Qdec_Ip_PhaseParamsType	mPhaseBConfig	Configuration of input phase B
Ftm_Qdec_Ip_CallbackCfgType	overflowCb	overflow callback
Ftm_Qdec_Ip_ClockSourceType	eSourceClock	FlexTimer Clock Source
Ftm_Qdec_Ip_ClockFilterPrescType	clockFilterPs	FlexTimer Clock Filter Prescaler

6.2.2.4 struct Ftm_Qdec_Ip_StateType

FTM Quadrature state (counter value and flags)

Definition at line 285 of file Ftm_Qdec_Ip_Types.h.

Data Fields

uint16	mCounter	FTM Counter value
boolean	mTimerOverflow	True if timer overflow occurred, False otherwise
boolean	mTimerOverflowDir	Timer Overflow Direction: False if there was an FTM counter decrement and FTM counter changes from its minimum value to its maximum value, True if there was an FTM counter increment and FTM counter changes from its maximum value to its minimum value
boolean	mCountDir	FTM Counter Direction in QDEC mode: False if counting direction is decreasing (counter decrement), True if counting direction is increasing (counter increment)

6.2.3 Types Reference

6.2.3.1 Ftm_Qdec_HwAddrType

```
typedef FTM_Type Ftm_Qdec_HwAddrType
```

FTM - Register Layout Typedef.

Definition at line 227 of file Ftm_Qdec_Ip_Types.h.

6.2.3.2 Ftm_Qdec_Ip_CallbackType

```
typedef void(* Ftm_Qdec_Ip_CallbackType) (uint8 callbackParam1)
```

Callback type for each instance.

Definition at line 230 of file Ftm_Qdec_Ip_Types.h.

6.2.4 Enum Reference

6.2.4.1 Ftm_Qdec_Ip_ClockSourceType

enum `Ftm_Qdec_Ip_ClockSourceType`

FlexTimer Clock Source Selection.

Enumerator

FTM_CLOCK_SOURCE_NONE	None use clock for FTM
FTM_CLOCK_SOURCE_SYSTEMCLK	System clock
FTM_CLOCK_SOURCE_FIXEDCLK	Fixed clock
FTM_CLOCK_SOURCE_EXTERNALCLK	External clock

Definition at line 113 of file `Ftm_Qdec_Ip_Types.h`.

6.2.4.2 Ftm_Qdec_Ip_ClockPrescType

enum `Ftm_Qdec_Ip_ClockPrescType`

FlexTimer Prescale Factor Selections bits (PS).

Selects one of 8 division factors for the clock source selected by CLKS. The new prescaler factor affects the clock source on the next FTM input clock cycle after the new value is updated into the register bits. This field is write protected. It can be written only when `MODE[WPDIS] = 1`. 000b - Divide by 1 001b - Divide by 2 010b - Divide by 4 011b - Divide by 8 100b - Divide by 16 101b - Divide by 32 110b - Divide by 64 111b - Divide by 128

Enumerator

FTM_CLOCK_DIVID_BY_1	Divide by 1
FTM_CLOCK_DIVID_BY_2	Divide by 2
FTM_CLOCK_DIVID_BY_4	Divide by 4
FTM_CLOCK_DIVID_BY_8	Divide by 8
FTM_CLOCK_DIVID_BY_16	Divide by 16
FTM_CLOCK_DIVID_BY_32	Divide by 32
FTM_CLOCK_DIVID_BY_64	Divide by 64
FTM_CLOCK_DIVID_BY_128	Divide by 128

Definition at line 136 of file `Ftm_Qdec_Ip_Types.h`.

6.2.4.3 Ftm_Qdec_Ip_ClockFilterPrescType

enum `Ftm_Qdec_Ip_ClockFilterPrescType`

FlexTimer Filter Prescaler.

The bits FLTPS selects the clock prescaler used in the FTM filters. Selects one of 16 division factors to affect channel input filters and quadrature decoder phase A and B inputs filters. Writing to the bits FLTPS has immediate effect.
 0000b - Divide by 1 0001b - Divide by 2 0010b - Divide by 3 0011b - Divide by 4 0100b - Divide by 5 0101b - Divide by 6 0110b - Divide by 7 0111b - Divide by 8 1000b - Divide by 9 1001b - Divide by 10 1010b - Divide by 11 1011b - Divide by 12 1100b - Divide by 13 1101b - Divide by 14 1110b - Divide by 15 1111b - Divide by 16

Enumerator

FTM_CLOCK_FILTER_PRESCALER_DIV1	Divide by 1
FTM_CLOCK_FILTER_PRESCALER_DIV2	Divide by 2
FTM_CLOCK_FILTER_PRESCALER_DIV3	Divide by 3
FTM_CLOCK_FILTER_PRESCALER_DIV4	Divide by 4
FTM_CLOCK_FILTER_PRESCALER_DIV5	Divide by 5
FTM_CLOCK_FILTER_PRESCALER_DIV6	Divide by 6
FTM_CLOCK_FILTER_PRESCALER_DIV7	Divide by 7
FTM_CLOCK_FILTER_PRESCALER_DIV8	Divide by 8
FTM_CLOCK_FILTER_PRESCALER_DIV9	Divide by 9
FTM_CLOCK_FILTER_PRESCALER_DIV10	Divide by 10
FTM_CLOCK_FILTER_PRESCALER_DIV11	Divide by 11
FTM_CLOCK_FILTER_PRESCALER_DIV12	Divide by 12
FTM_CLOCK_FILTER_PRESCALER_DIV13	Divide by 13
FTM_CLOCK_FILTER_PRESCALER_DIV14	Divide by 14
FTM_CLOCK_FILTER_PRESCALER_DIV15	Divide by 15
FTM_CLOCK_FILTER_PRESCALER_DIV16	Divide by 16

Definition at line 171 of file `Ftm_Qdec_Ip_Types.h`.

6.2.4.4 Ftm_Qdec_Ip_EncodingModeType

enum `Ftm_Qdec_Ip_EncodingModeType`

Encoding mode used by FTM in the QDEC mode.

Enumerator

FTM_MODE_PHASE_ENCODE	Phase A and phase B encoding mode
FTM_MODE_COUNT_AND_DIR_ENCODE	Count and direction encoding mode

Definition at line 194 of file Ftm_Qdec_Ip_Types.h.

6.2.4.5 Ftm_Qdec_Ip_PhasePolarityType

enum `Ftm_Qdec_Ip_PhasePolarityType`

Quadrature phase polarities, normal or inverted polarity.

Enumerator

FTM_PHASE_NORMAL	Phase input signal is not inverted before identifying the rising and falling edges of this signal
FTM_PHASE_INVERT	Phase input signal is inverted before identifying the rising and falling edges of this signal

Definition at line 203 of file Ftm_Qdec_Ip_Types.h.

6.2.4.6 Ftm_ModeType

enum `Ftm_ModeType`

Qdec mode type include: QDEC_MODE_NORMAL and QDEC_MODE_SLEEP.

Enumerator

FTM_QDEC_MODE_NORMAL	Normal operation, all used interrupts are enabled according to the notification requests.
FTM_QDEC_MODE_SLEEP	Reduced power operation.

Definition at line 214 of file Ftm_Qdec_Ip_Types.h.

6.2.5 Function Reference

6.2.5.1 Ftm_Qdec_Ip_Init()

```
void Ftm_Qdec_Ip_Init (
    uint8 module,
    const Ftm_Qdec_Ip_ConfigType * config )
```

Configures the quadrature mode and starts measurement.

Parameters

in	<i>module</i>	FTM hardware instance number.
in	<i>config</i>	Configuration structure(Quadrature Decoder Mode, polarity for both phases, minimum and maximum value for the counter, filter configuration).

Returns

No return.

6.2.5.2 Ftm_Qdec_Ip_Deinit()

```
void Ftm_Qdec_Ip_Deinit (
    uint8 module )
```

De-activates the Quadrature Decoder Mode.

Parameters

in	<i>module</i>	FTM hardware instance number.
----	---------------	-------------------------------

Returns

No return.

6.2.5.3 Ftm_Qdec_Ip_GetState()

```
Ftm_Qdec_Ip_StateType Ftm_Qdec_Ip_GetState (
    uint8 module )
```

Return the current quadrature decoder state (counter value, timer overflow, TOF direction and counting direction flags)

Parameters

in	<i>module</i>	FTM hardware instance number.
----	---------------	-------------------------------

Returns

Structure with the state of FTM module

6.2.5.4 Ftm_Qdec_Ip_CounterState()

```
void Ftm_Qdec_Ip_CounterState (
    const uint8 module )
```

Set state of counter.

Parameters

in	<i>module</i>	FTM hardware number.
----	---------------	----------------------

Returns

void

6.2.5.5 Ftm_Qdec_Ip_ResetState()

```
void Ftm_Qdec_Ip_ResetState (
    const uint8 module )
```

Reset state of module to default value.

Parameters

in	<i>module</i>	FTM hardware number.
----	---------------	----------------------

Returns

void

6.2.5.6 Ftm_Qdec_Ip_SetMode()

```
void Ftm_Qdec_Ip_SetMode (
    const uint8 module,
    Ftm_ModeType mode )
```

Set mode of Qdec module.

Parameters

in	<i>module</i>	FTM hardware number.
in	<i>mode</i>	Given mode: QDEC_NORMAL_MODE or QDEC_SLEEP_MODE.

Returns

void

6.2.6 Variable Documentation

6.2.6.1 scpFtmBase

```
Ftm_Qdec_HwAddrType* const scpFtmBase[FTM_INSTANCE_COUNT] [extern]
```

Array with base addresses for FTM instances available on platform.

6.2.6.2 gQdecFtmInitState

```
boolean gQdecFtmInitState [extern]
```

Quadrature ftm initialization state variable.

6.2.6.3 Ftm_Qdec_Ip_pOverflowIrqCallback

`Ftm_Qdec_Ip_CallbackCfgType` `Ftm_Qdec_Ip_pOverflowIrqCallback[FTM_INSTANCE_COUNT]` [extern]

Array with notification handlers for all configurable instances.

How to Reach Us:

Home Page:

nxp.com

Web Support:

nxp.com/support

Limited warranty and liability - Information in this document is believed to be accurate and reliable. However, NXP Semiconductors does not give any representations or warranties, expressed or implied, as to the accuracy or completeness of such information and shall have no liability for the consequences of use of such information. NXP Semiconductors takes no responsibility for the content in this document if provided by an information source outside of NXP Semiconductors.

In no event shall NXP Semiconductors be liable for any indirect, incidental, punitive, special or consequential damages (including - without limitation - lost profits, lost savings, business interruption, costs related to the removal or replacement of any products or rework charges) whether or not such damages are based on tort (including negligence), warranty, breach of contract or any other legal theory.

Notwithstanding any damages that customer might incur for any reason whatsoever, NXP Semiconductors' aggregate and cumulative liability towards customer for the products described herein shall be limited in accordance with the Terms and conditions of commercial sale of NXP Semiconductors.

Right to make changes - NXP Semiconductors reserves the right to make changes to information published in this document, including without limitation specifications and product descriptions, at any time and without notice. This document supersedes and replaces all information supplied prior to the publication hereof.

Suitability for use in automotive applications - This NXP product has been qualified for use in automotive applications. If this product is used by customer in the development of, or for incorporation into, products or services (a) used in safety critical applications or (b) in which failure could lead to death, personal injury, or severe physical or environmental damage (such products and services hereinafter referred to as "Critical Applications"), then customer makes the ultimate design decisions regarding its products and is solely responsible for compliance with all legal, regulatory, safety, and security related requirements concerning its products, regardless of any information or support that may be provided by NXP. As such, customer assumes all risk related to use of any products in Critical Applications and NXP and its suppliers shall not be liable for any such use by customer. Accordingly, customer will indemnify and hold NXP harmless from any claims, liabilities, damages and associated costs and expenses (including attorneys' fees) that NXP may incur related to customer's incorporation of any product in a Critical Application.

Applications - Applications that are described herein for any of these products are for illustrative purposes only. NXP Semiconductors makes no representation or warranty that such applications will be suitable for the specified use without further testing or modification.

Customers are responsible for the design and operation of their applications and products using NXP Semiconductors products, and NXP Semiconductors accepts no liability for any assistance with applications or customer product design. It is customer's sole responsibility to determine whether the NXP Semiconductors product is suitable and fit for the customer's applications and products planned, as well as for the planned application and use of customer's third party customer(s). Customers should provide appropriate design and operating safeguards to minimize the risks associated with their applications and products.

NXP Semiconductors does not accept any liability related to any default, damage, costs or problem which is based on any weakness or default in the customer's applications or products, or the application or use by customer's third party customer(s). Customer is responsible for doing all necessary testing for the customer's applications and products using NXP Semiconductors products in order to avoid a default of the applications and the products or of the application or use by customer's third party customer(s). NXP does not accept any liability in this respect.

Terms and conditions of commercial sale - NXP Semiconductors products are sold subject to the general terms and conditions of commercial sale, as published at <http://www.nxp.com/profile/terms>, unless otherwise agreed in a valid written individual agreement. In case an individual agreement is concluded only the terms and conditions of the respective agreement shall apply. NXP Semiconductors hereby expressly objects to applying the customer's general terms and conditions with regard to the purchase of NXP Semiconductors products by customer.

Export control - This document as well as the item(s) described herein may be subject to export control regulations. Export might require a prior authorization from competent authorities.

Translations — A non-English (translated) version of a document, including the legal information in that document, is for reference only. The English version shall prevail in case of any discrepancy between the translated and English versions.

Security — Customer understands that all NXP products may be subject to unidentified vulnerabilities or may support established security standards or specifications with known limitations. Customer is responsible for the design and operation of its applications and products throughout their lifecycles to reduce the effect of these vulnerabilities on customer's applications and products. Customer's responsibility also extends to other open and/or proprietary technologies supported by NXP products for use in customer's applications. NXP accepts no liability for any vulnerability. Customer should regularly check security updates from NXP and follow up appropriately.

Customer shall select products with security features that best meet rules, regulations, and standards of the intended application and make the ultimate design decisions regarding its products and is solely responsible for compliance with all legal, regulatory, and security related requirements concerning its products, regardless of any information or support that may be provided by NXP.

NXP has a Product Security Incident Response Team (PSIRT) (reachable at <mailto:PSIRT@nxp.com>) that manages the investigation, reporting, and solution release to security vulnerabilities of NXP products.

NXP B.V. - NXP B.V. is not an operating company and it does not distribute or sell products.